タスク分担：フロントエンド

分担の前提

企画書のMVPを対象とし、4人とも16ポイントにそろえます。1ポイントは小さな作業の相対的な目安で、時間の確約ではありません。設計・実装・担当機能の確認を含みます。動画・並び替え・画像編集・撮影日時保持などの追加機能は対象外です。

佐藤さん｜画面・閲覧UI・開発基盤［合計16］

① 開発基盤・ルーティング・共通UI［4］
React＋TypeScript環境、4画面のルート、ボタン・モーダル・スマホ用レイアウトを整える。
② トップ・招待画面［3］
ルーム作成UI、QR表示、URLコピー、アルバムへの遷移を作る。ルーム処理は志鳥さんのAPIを呼ぶ。
③ アルバム・詳細画面の表示［5］
写真グリッド、拡大表示、前後移動、読み込み中・空・エラー表示、期限切れ画面を作る。
④ モバイル表示確認・統合・起動手順［4］
4画面を接続し、スマホ表示と遷移を確認。ビルド設定とREADMEを整える。

佐藤さんの所有ファイル

frontend/src/ の app/、pages/、components/、styles/、main.tsx、features/photos/components/PhotoGrid.tsx・PhotoViewer.tsx。
frontend/ の package.json・ロックファイル・Vite／TS／ESLint設定・index.html、およびルートのREADME.md・.gitignore。

雨谷さん｜写真操作・選択・端末保存［合計16］

① 写真取得・選択状態［4］
写真一覧の取得、複数選択、全選択・解除、長押し、一覧と詳細での選択共有を作る。
② 投稿操作［4］
複数ファイル選択、投稿進捗、失敗表示・再試行を作る。容量等の確定判定はバックエンドで行う。
③ 単体・一括保存と端末検証［5］
Web Share APIの利用可否を確認し、非対応時のダウンロード／ZIP保存を用意。HEIC保存と端末ごとの差を確認する。
④ 自分の写真の削除操作［3］
削除UI・確認・一覧更新を作り、選択・投稿・保存・削除の一連の操作を確認する。

雨谷さんの所有ファイル

frontend/src/features/photos/ の api.ts、hooks/、utils/、types.ts、およびUploadPanel.tsx・SelectionToolbar.tsx・DeletePhotoDialog.tsx等の操作用components。佐藤さん所有のPhotoGrid.tsx・PhotoViewer.tsxは編集しない。

タスク分担：バックエンド

志鳥さん｜DB・匿名認証・ルーム・権限［合計16］

① DB設計・共通契約・生成型［4］
rooms・photos等のスキーマ、写真レコードとStorageパスの関係を設計。API仕様とDB生成型を管理する。
② 匿名認証・参加・ルーム作成［4］
匿名セッション、ルーム作成、URLからの参加、24時間の有効期限を実装する。frontend側の認証・ルームAPIも担当する。
③ DBのRLS・権限・期限判定［5］
参加者の閲覧・投稿、自分の写真だけの削除、期限切れのアクセス拒否を実装。匿名ユーザーを分けて権限を確認する。
④ Supabase基盤・接続確認［3］
ローカル設定、環境変数の見本、DBテストと接続手順を整え、佐藤さんとルーム作成〜参加を確認する。

志鳥さんの所有ファイル

frontend/src/ の features/auth/、features/rooms/、lib/supabase.ts、types/database.ts。
backend/supabase/ の config.toml、seed.sql、functions/_shared/、functions/create-room/、functions/join-room/、tests/db/。
DB・RLS用の新規migrations、docs/api.md・database.md・development.md、frontend/.env.example。

斎藤さん｜画像・容量制限・整合性・期限削除［合計16］

① Storage・画像アップロード［5］
非公開バケット、Storage権限、投稿用の処理を実装。JPEG・PNG・HEICの受入条件を確認し、HEICプレビューの扱いを雨谷さんと決める。
② 枚数・容量制限と失敗時の整合性［4］
1ルーム100枚・500MBをサーバー側で保証。同時投稿時の予約／確定と、投稿失敗時の予約・ファイルの後片付けを実装する。
③ 写真削除・ダウンロード［3］
投稿者権限を確認してStorageとDBの両方を削除。期限内だけダウンロードできる仕組みを作る。
④ 期限切れ削除・再試行・検証［4］
Cronと削除処理を実装し、24時間経過後にファイルとルームを削除。途中失敗時の再実行、期限境界、残存ファイルを確認する。

斎藤さんの所有ファイル

backend/supabase/functions/ の upload-photo/、delete-photo/、download-photos/、cleanup-expired/。
Storage・Cron・容量管理用の新規migrations、tests/storage/・tests/functions/、docs/storage.md。_shared/の変更は志鳥さんに依頼する。

バックエンド2人の境界

photos等の基本テーブルとDBのRLSは志鳥さん、Storageのポリシー・容量予約のSQL関数・Cronは斎藤さんが担当。既存のSQLを共同編集せず、変更ごとに新しいmigrationを追加する。共通テーブルに関わる変更は設計を先に合意する。

衝突を減らすルールと着手順

1｜実装前に接続仕様を決める

4人で短く打ち合わせ、志鳥さんがdocs/api.mdに記録する。ルートは /、/rooms/:roomId/invite、/rooms/:roomId、/rooms/:roomId/photos/:photoId を案とする。
APIの入力・戻り値、roomId／photoId、期限、写真サイズ・形式、エラーコードを決める。ルームURLの推測対策と、参加者をどう判定するかも合意する。

2｜画面と操作はpropsで接続する

PhotoGrid・PhotoViewerは佐藤さんが作る表示部品とし、photos・selectedIds・onSelect・onOpen等を受け取る。選択状態と投稿・保存・削除の処理は雨谷さんのhooksに置く。
ページへの組み込みは佐藤さんだけが行う。雨谷さんはhooksと操作部品の使い方を渡す。志鳥さんは認証・ルームAPI、斎藤さんは画像APIの呼出仕様を渡す。

3｜共有ファイルは主担当に変更を集約する

依存ライブラリとロックファイル、フロント設定は佐藤さん。Supabase設定、共通サーバー処理、DB生成型、API仕様は志鳥さん。
担当外のファイルは無断で変更せず、必要な変更を主担当に伝える。各担当は所有ファイル内に必要なテストを置く。追加のroot設定は作成時に管理担当を決める。

4｜migrationとGitの運用

migrationのファイル名は作成日時＋用途で一意にする。取り込み済みのmigrationは修正せず、追加ファイルで変更する。共通テーブルに関わるSQLは取り込み順も確認する。
ブランチはfeature/機能名で分け、1機能を小さなPRにする。作業前とPR前にmainの更新を取り込み、担当範囲を越える差分がないか確認する。同じファイルを触らなくても、APIやDB仕様の不一致は起こるため接続確認を行う。

5｜並行開発の進め方

準備：佐藤さんがフロント基盤、志鳥さんがAPI・DBの土台を作る間、雨谷さんは端末保存、斎藤さんはHEIC・Storage・同時投稿制限を検証する。
実装：接続仕様を確定し、佐藤さんは仮データで画面、雨谷さんは操作hooks、志鳥さんはルーム・認証、斎藤さんは画像処理を並行して作る。
結合：佐藤＋志鳥で作成・参加、雨谷＋斎藤で投稿・保存・削除を確認。最後に4人で別端末から全体を確認する。

6｜作業量の見直しとMVPの完了条件

16ポイントずつでも実際の負荷は変わるため、週1回、残り作業を見直す。保存・HEIC・同時投稿は不確実性が高く、余裕がある人は独立した調査や動作確認を引き取る。ファイル担当を移す場合は、先に引継ぎを宣言する。
完了条件：登録なしで作成・参加でき、写真の投稿・閲覧・複数選択・単体／一括保存ができる。自分の写真だけ削除でき、枚数・容量・期限が守られ、24時間後の自動削除が動く。対応端末・画像形式の検証結果を記録する。